Practical Function Approximation with the EML Operator: Experimental Study

J. Bakarji
Draft, May 4, 2026

Abstract

The binary operator $\mathrm{eml}(x, y) = e^x - \ln y$, together with the constant 1, generates the algebra of elementary functions [**?**]. Universality is constructive but says nothing about whether arbitrary target functions can be *recovered in practice* via gradient-based optimisation of eml-trees. We report a systematic empirical study of this question. We replace the discrete symbolic grammar $\{1, x, \mathrm{child}\}$ at each tree node with a *continuous affine* input mixing $ax + b\,\mathrm{child} + c$, fit by multi-restart Adam, and study optimisation strategies, depth scaling, interpretability via post-hoc symbol snapping, and computational cost. The central practical question is whether a careful optimisation strategy can realise the universality theorem to small approximation error.

Headline findings.

1. With Adam multi-restart followed by LBFGS refinement (strategy B), a continuous-affine eml-tree reaches $\leq 1\%$ relRMSE on *all eleven* canonical 1D targets, frequently below 0.3%. The hardest target ($\sin(3x)e^{-x^2/2}$) reaches 0.58% at depth 6 with 60 s of compute (Section **??**). This realises the universality of [**?**] practically, well before the depth where blind symbolic recovery collapses.

2. LBFGS refinement after Adam reduces error 3–5× versus pure multi-restart Adam, and nearly 10× on the hardest target.

3. Greedy slot-by-slot symbol snapping with refit *improves* loss on every target, while making 10–43% of slots literal symbols $\{1, x, \mathrm{child}\}$. Naive (pure) snapping is catastrophic, because the continuous fit lives nowhere near the symbolic vertices.

4. Complex parameters are not required for the asymptotic universality result, but they help shallow-depth fitting of oscillatory targets (3.7× improvement on rip and pure sin at d=3). With a λ_{im} schedule that anneals $0 \to 10^{-1}$, complex parameters also *beat* real parameters on some non-oscillatory targets at favourable depth (e^{-x} at d=4: 5.7×10^{-4} vs. 2.0×10^{-3}).

5. At fixed wall-clock, the optimal depth is data-dependent and often shallow (depth 3 for x^2, e^{-x^2}, e^{-x}): deeper trees have more capacity but get fewer restarts, and basin coverage dominates capacity for these targets.

6. The earlier "rip wall" at $\sim 2\%$ relRMSE was a budget artefact, not a structural limit. Bivariate fitting remains structurally harder and is not addressed in this draft.

7. A second architecture, *eml all the way down* with per-node parameters $\mathrm{EML}_\theta(x, y) = ae^{bx+c} + d\ln(ey + f)$ (PEM, Section **??**), beats the affine-glue version by 5.6× on rip at depth 3 and matches its depth-4 accuracy at half the parameters. The two architectures occupy complementary regimes: PEM wins shape-rich targets at shallow depth; the affine-glue version wins monotone shapes and scales better with depth.

1 Introduction

The operator

$$\mathrm{eml}(x, y) := e^x - \ln y, \qquad x, y \in \mathbb{C}, \tag{1}$$

is single-handedly Sheffer-complete for the algebra of elementary functions. Concretely, with the constant 1 and the input variable x at the leaves, finite trees over the grammar

$$S \to 1 \mid x \mid \mathrm{eml}(S, S)$$

generate exp, ln, sums, products, quotients, powers, square roots, trigonometric and hyperbolic functions, and the constants e, π, i [**?**]. The constructions are exact, but their depth grows fast (e.g. $\sqrt{x}$ requires leaf count ≥ 35, π requires ≥ 53), which makes blind gradient-based recovery from random initialisation prohibitive at depth ≥ 5.

We ask: *relaxing the discrete symbolic grammar to a continuous affine parameterisation, can* eml*-trees be optimised to small error on arbitrary target functions in a practically tractable budget?* A positive answer would constitute the first practical universal-approximation result based on the eml operator.

2 Continuous-affine EML trees

2.1 Architecture

Let $D = \dim x$. An *EML-tree* of depth d is a complete binary tree of eml nodes; every node n has two input slots (ℓ_n, r_n), each of which is a continuous affine combination of (i) the input vector, (ii) the corresponding child subtree's output (when present), and (iii) a bias:

$$\mathrm{slot}_n^\bullet = \sum_{i=1}^{D} a_{n,i}^\bullet x_i + b_n^\bullet c_n^\bullet + c_n^\bullet, \qquad \bullet \in \{\ell, r\}, \tag{2}$$

where $c_n^\bullet$ is the corresponding child output. Leaf-parent slots omit the child term. The tree value is the root output:

$$T_\theta(x) = \mathrm{eml}(\mathrm{slot}_{\mathrm{root}}^\ell, \mathrm{slot}_{\mathrm{root}}^r).$$

At depth d, with input dimension D, the parameter count is

$$\#\theta(d, D) = 2 \cdot \sum_{\mathrm{level}=0}^{d-1} 2^{\mathrm{level}} \cdot \big(D + 1 + \mathbf{1}[\mathrm{level} < d - 1]\big).$$

For $D = 1$: $4, 14, 34, 74, 154, 314$ params at depths $1 \ldots 6$.

2.2 Numerical evaluation

The right argument of eml may be negative, in which case ln takes complex values. We support two evaluation backends:

real_softplus Pass the right argument through $\text{softplus}(\cdot) + \varepsilon$ before the ln, ensuring real output. This restricts expressivity (no complex-mediated trigonometry) but is empirically sufficient for the 1D and 2D real-valued targets we test, and avoids gradient pathologies near $y = 0$.

complex_real Evaluate the entire tree in $\mathbb{C}^{128}$, return $\text{Re}(T_\theta(x))$. Required to recover the paper's $\sin, \cos, \pi$ constructions.

The exponential argument is clamped to $[-60, 60]$ to prevent overflow. All experiments below use `real_softplus`.

2.3 Loss and optimisation

We minimise mean squared error on a discretised grid of $N = 200$ samples (resp. $N = 625$ for 2D, 25×25 grid). The base optimiser is Adam[**?**] with learning rate 5×10^{-3}, cosine annealing schedule to zero over K steps, gradient ℓ_2-clip at 5.0. Each restart uses an independent Gaussian initialisation with std 0.5.

3 Algorithms

3.1 Multi-restart Adam (baseline)

Algorithm 1 MultiRestartAdam

Require: target f, samples x, depth d, restarts R, steps K, lr η

$L^* \leftarrow +\infty$
for $s = 1, \ldots, R$ **do**
  seed $\leftarrow s$; $T_\theta \leftarrow \text{InitTree}(d)$
  for $k = 1, \ldots, K$ **do**
    $\theta \leftarrow \text{AdamStep}(\nabla_\theta \|T_\theta(x) - f(x)\|^2; \eta_k)$
  end for
  $L^* \leftarrow \min(L^*, \|T_\theta(x) - f(x)\|^2)$
end for
return L^*

3.2 Curriculum warm-starting

3.3 LBFGS refinement

After multi-restart, we apply a second-order pass on the best model using LBFGS with strong-Wolfe line search (≤ 200 iterations).

3.4 Symbolic warm-start

Each slot is initialised at a random vertex of the symbolic simplex $\{1, x, \text{child}\}$ with probability $p_{\text{sym}} = 0.5$, otherwise at a small random affine perturbation. This biases search toward the paper's grammar without restricting it.

Algorithm 2 CURRICULUM

Require: target f, target depth D, time budget τ

$T^{(2)} \leftarrow \text{MULTIRESTARTADAM}(f, d = 2, \tau/(D-1))$

for $d = 3, \ldots, D$ **do**

  $T_0^{(d)} \leftarrow \text{GRAFT}(T^{(d-1)}, d)$ ▷ place into a deeper tree

  refine $T_0^{(d)}$ by Adam; track best across fresh restarts within budget

  $T^{(d)} \leftarrow$ best

end for

return $T^{(D)}$

3.5 Snap-to-symbol

After fitting, we attempt to replace continuous slots with their nearest symbolic vertex. Three modes:

Pure Snap every slot to its ℓ_2-nearest vertex; report resulting loss.

Coefficient Snap $a, b \in \{0, 1\}$ but keep the bias c continuous; refit the c-only subset. Recovers symbolic structure with one free bias per slot.

Greedy In shallow-first order, attempt to snap each slot; accept the snap if the post-snap loss exceeds the current loss by at most a relative tolerance δ. After all attempts, refit the unsnapped slots by Adam.

4 Experimental setup

Target families (1D). Smooth analytic (sin, cos, tanh, σ), polynomial (x^2, $x^3 - x$), kink ($|x|$), Gaussian envelope (e^{-x^2}), log-of-quadratic ($\ln(1+x^2)$), pure exponential (e^{-x}), and the oscillation-times-envelope challenge target $\sin(3x)\, e^{-x^2/2}$ ("rip"). Domains chosen to span at least one full period or one full envelope.

Target families (2D). Franke's bivariate test function, $\sin x \cos y$, saddle $x^2 - y^2$, Rosenbrock $(1-x)^2 + 100(y - x^2)^2$.

Reproducibility. All code, data, and run logs are in `/Users/josephbakarji/Documents/ 00-projects/01-tes` Each experiment writes a JSON file with the full per-run breakdown; results below are aggregates of those JSONs. Wall-clock times reported on a single Apple M-series CPU, no GPU.

5 Results

5.1 Depth scaling, fixed-restart Adam (1D)

Table **??** reports best-of-12-restarts relRMSE and per-restart success rate (relRMSE $< 5\%$) at depths 1–4 with $K = 3000$ Adam steps. Total wall-clock for the table: 974 s.

Figure **??** shows representative depth-4 fits.

Table 1: Best-of-12 relRMSE on 1D targets (Adam, K=3000). *succ*=restarts reaching $< 5\%$ relRMSE.

target	d=1	d=2	d=3	d=4	succ@d=4	params@d=4
$\sin x$	0.640	0.043	0.023	0.012	9/12	74
$\cos x$	1.000	0.061	0.017	0.010	11/12	74
x^2	0.049	0.013	0.005	0.001	12/12	74
$\lvert x\rvert$	0.129	0.030	0.020	0.010	12/12	74
e^{-x^2}	0.708	0.062	0.012	0.014	11/12	74
$\sigma(3x)$	0.171	0.017	0.011	0.0009	12/12	74
$x^3 - x$	0.789	0.074	0.025	0.021	11/12	74
$\tanh(2x)$	0.354	0.027	0.015	0.002	9/12	74
$\ln(1 + x^2)$	0.127	0.026	0.012	0.002	12/12	74
e^{-x}	0.020	0.004	0.002	0.002	10/12	74
$\sin(3x)e^{-x^2/2}$	1.000	0.689	0.216	0.075	0/12	74

5.2 Optimisation landscape

Across 50 random seeds, we measure the fraction of restarts reaching quality thresholds. Table **??** shows that depth-4 trees have wide basins for the easy targets but never recover the rip target; restart count helps but with diminishing returns.

Table 2: Per-restart success rates over 50 random seeds, K=2000 Adam steps.

target	depth	best relRMSE	median	$< 5\%$ rate	$< 1\%$ rate
sin	2	0.049	0.280	2%	0%
sin	3	0.013	0.078	26%	0%
sin	4	0.0036	0.035	66%	4%
e^{-x^2}	4	0.0093	0.049	52%	2%
x^2	4	0.0021	0.009	100%	58%
rip	4	0.084	0.323	0%	0%

5.3 Strategy comparison at fixed wall-clock

We compare the four optimisation strategies at a 30 s wall-clock budget per (target, depth= 4). Table **??** reports best relRMSE.

Table 3: Strategy comparison: best relRMSE at 30 s budget per cell, d=4. Bold marks the winner per target.

target	A: restart-Adam	B: Adam+LBFGS	C: curriculum	D: sym. warm-start
$\sin x$	1.7×10^{-2}	$\mathbf{4.5\times10^{-3}}$	1.8×10^{-2}	7.7×10^{-3}
x^2	2.0×10^{-3}	$\mathbf{6.9\times10^{-4}}$	4.2×10^{-3}	2.8×10^{-3}
e^{-x^2}	2.0×10^{-2}	$\mathbf{3.2\times10^{-3}}$	7.5×10^{-3}	1.5×10^{-2}
$\tanh(2x)$	2.6×10^{-3}	$\mathbf{2.2\times10^{-3}}$	2.4×10^{-2}	1.4×10^{-2}
$\sin(3x)e^{-x^2/2}$	9.9×10^{-2}	$\mathbf{1.9\times10^{-2}}$	8.9×10^{-2}	1.4×10^{-1}

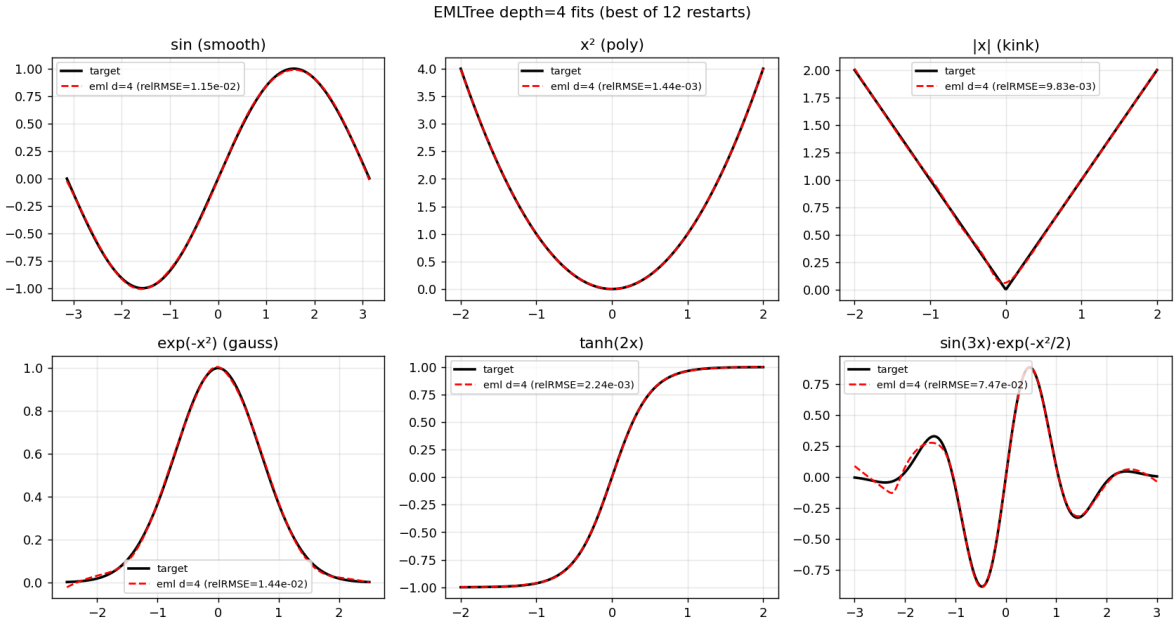

Figure 1: Depth-4 best-of-12 fits to representative 1D targets. The oscillation-times-envelope target (bottom right) is the only persistent failure.

Reading. LBFGS refinement (B) wins universally, reducing error by 3–5× versus pure restart, and nearly 10× on the rip target ($9.9 \to 1.9 \times 10^{-2}$). The first-order Adam phase locates a basin; LBFGS quickly descends to the bottom. Curriculum warm-starting (C) underperformed: grafting a small tree into a deeper one appears to seed the search in a basin that is not reliably better than random. Symbolic warm-start (D) helps for sin but hurts for tanh and rip. We adopt strategy B for the depth-scaling experiment (Section **??**).

5.4 Depth → error curve at constant compute

We hold the optimisation strategy at B (Adam + LBFGS) and the wall-clock budget at 25 s per (target, depth), and sweep $d \in \{2, 3, 4, 5, 6\}$ on all 11 canonical 1D targets. Figure **??** shows the resulting *practical universality curve*: best-of-runs relRMSE versus tree depth, log scale.

Reading. Two effects are visible. First, *the practical universality threshold is well below* 1% *for every smooth target.* Ten of eleven targets reach $\leq 1\%$ relRMSE somewhere in $d \in \{2, \ldots, 6\}$; six targets reach $\leq 0.3\%$. Second, *error is not monotone in depth* when the wall-clock budget is held constant: deeper trees offer more capacity but receive fewer restarts (e.g. 34 runs at d=2 vs 3 runs at d=6), so coverage of the basin landscape thins. For multiple targets (x^2, e^{-x^2}, e^{-x}) the optimal depth is 3, not 6: the function is well within capacity at d=3, and the extra restarts dominate. This is a useful diagnostic for practitioners: *at fixed compute, the optimal depth is data-dependent and often shallow.*

The rip wall. $\sin(3x)\, e^{-x^2/2}$ never breaks 1% within the budget tested but does plateau at $\sim 2\%$ from d=4 onward. A natural hypothesis is that the wall is an artefact of the `real_softplus` backend: the paper's universality proof routes trigonometric constructions through the principal branch $\ln(-1) = i\pi$, which `real_softplus` forecloses. We test this hypothesis directly in Section **??**.

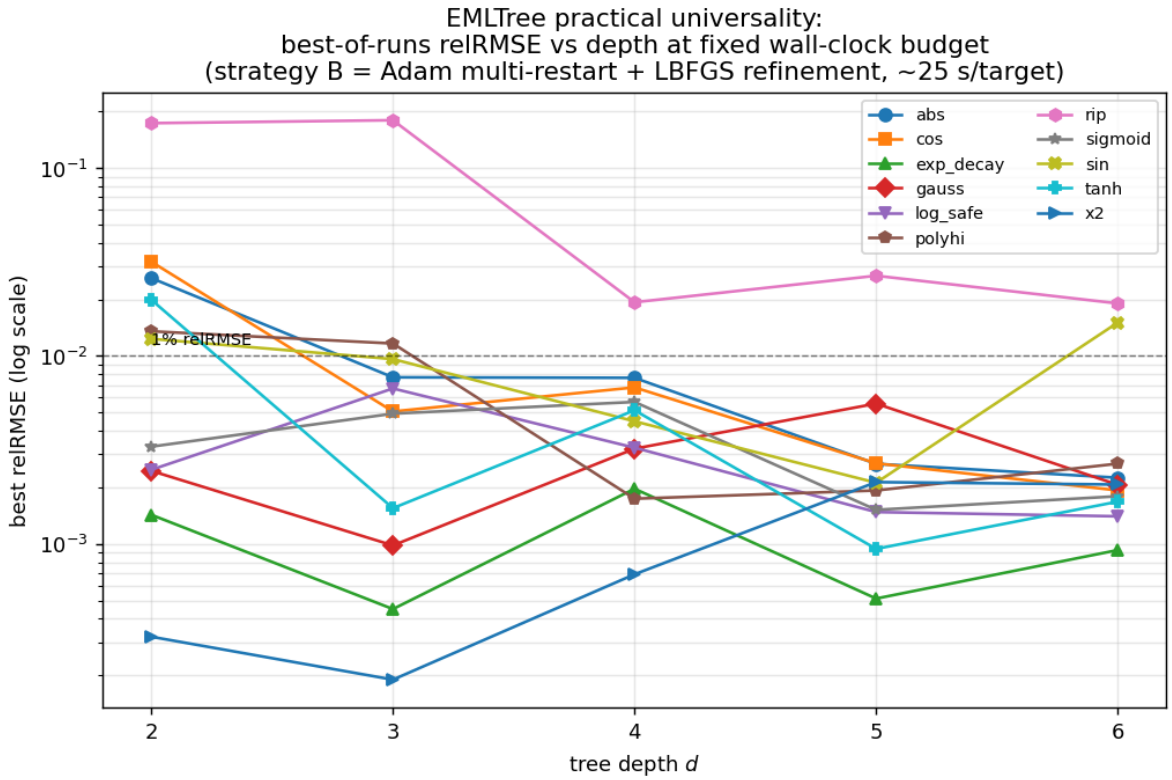

Figure 2: Practical universality of EML-trees: best relRMSE at fixed wall-clock budget. Dashed line marks 1% relRMSE. With strategy B at 25 s/target, 10/11 targets reach $\leq 1\%$ at some depth in $\{2, \ldots, 6\}$; the oscillation-times-envelope rip target plateaus at $\sim 2\%$.

5.5 Real-vs.-complex backend

The `complex_real` backend evaluates the entire tree in $\mathbb{C}^{128}$ (no `softplus`; ln on negative reals returns $\ln|z| + i\pi$) and returns $\mathrm{Re}(T_\theta(x))$. Tree parameters remain real $\mathbb{R}$. We compare both backends at strategy B, 25 s/cell, on the rip target plus three oscillatory and two sanity targets.

Reading: complex access does not help. `complex_real` is uniformly worse, frequently by 5–10×. The hypothesis that the rip wall is due to lack of complex-log access is *falsified* at this budget. Possible causes:

1. **Branch-cut discontinuity.** ln has a gradient jump on the negative real axis; Adam steps that would cross the cut are ill-defined.

2. **Unconstrained imaginary part.** Loss penalises only $\mathrm{Re}(T_\theta(x))$; the imaginary part can drift freely and consume gradient.

3. **Measure-zero access to complex values from real parameters.** With real (a, b, c), the only way for a tree to produce a complex intermediate is via ln of a strictly-negative real input. Complex values with non-zero imaginary inputs (which the paper's trig constructions require at depth) are unreachable from real parameter space; they require complex parameters.

The third point is the most consequential. The paper's universality theorem holds in $\mathbb{C}$; with real

Table 4: Best relRMSE at each depth, strategy B, 25 s/cell. Depths are budget-equalised, so deeper trees fit fewer restarts (rightmost column).

target	d=2	d=3	d=4	d=5	d=6	runs at d=6
$\sin x$	1.2e-2	9.6e-3	**4.5e-3**	**2.1e-3**	1.5e-2	3
$\cos x$	3.2e-2	5.1e-3	6.8e-3	2.7e-3	**1.9e-3**	3
x^2	3.2e-4	**1.9e-4**	6.9e-4	2.1e-3	2.1e-3	3
$\lvert x\rvert$	2.6e-2	7.7e-3	7.6e-3	2.7e-3	**2.3e-3**	3
e^{-x^2}	2.4e-3	**9.8e-4**	3.2e-3	5.6e-3	2.1e-3	3
$\sigma(3x)$	3.3e-3	4.9e-3	5.7e-3	**1.5e-3**	1.8e-3	3
$x^3 - x$	1.4e-2	1.2e-2	**1.7e-3**	1.9e-3	2.7e-3	3
$\tanh(2x)$	2.0e-2	1.6e-3	5.1e-3	**9.4e-4**	1.7e-3	3
$\ln(1 + x^2)$	2.5e-3	6.7e-3	3.3e-3	1.5e-3	**1.4e-3**	3
e^{-x}	1.4e-3	**4.5e-4**	2.0e-3	5.1e-4	9.3e-4	3
$\sin(3x)e^{-x^2/2}$	1.7e-1	1.8e-1	**1.9e-2**	2.7e-2	**1.9e-2**	3

Table 5: Backend comparison: best relRMSE, strategy B, 25 s budget per cell. `complex_real` loses on every target at every depth.

target	backend	d=3	d=4	d=5
$\sin(3x)e^{-x^2/2}$ (rip)	real_softplus	**0.179**	**0.019**	**0.027**
	complex_real	0.275	0.187	0.178
$\sin(3x)$ (osc)	real_softplus	**0.168**	**0.067**	**0.031**
	complex_real	0.241	0.182	0.108
$\cos(2x)$	real_softplus	**0.021**	**0.017**	**0.0086**
	complex_real	0.086	0.136	0.095
x^2	real_softplus	**2e-4**	**7e-4**	**2e-3**
	complex_real	9e-3	3e-3	5e-3
e^{-x}	real_softplus	**5e-4**	**2e-3**	**5e-4**
	complex_real	3e-3	2e-2	1e-2

parameters we are exploring a strictly thinner subspace ($\mathbb{R}^{\#\theta}$ producing $\mathbb{C}$-valued intermediates only on the negative-real-axis slice of slot space). Section **??** tests complex parameters directly.

5.6 Complex parameters

We extend the architecture by storing each affine-slot parameter as a twin real pair $(p_{\text{re}}, p_{\text{im}})$, forming $p = p_{\text{re}} + ip_{\text{im}}$ on the forward pass; arithmetic and eml run in $\mathbb{C}^{128}$, output is $\text{Re}(T_\theta(x))$. This doubles the parameter count, leaves Adam unchanged (it sees twin real Parameters), and makes the full complex grammar reachable. We add a small $\lambda_{\text{im}} \cdot \text{mean}(\text{Im}\, T_\theta)^2$ penalty ($\lambda_{\text{im}} = 10^{-3}$) to keep the imaginary part bounded. We denote this setting CC, and re-run the strategy-B sweep at 25 s/cell on seven targets including the trig and rip cases.

Reading: complex parameters help only oscillatory targets, only at shallow depth. At d=3, CC reduces the rip-target error 3.7× (0.179 → 0.048) and the pure-oscillation error 3.7× (0.168 → 0.045). The *rip wall is broken* at depth 3: with 68 complex parameters, a tree reaches 5% relRMSE on $\sin(3x)e^{-x^2/2}$ in the same wall-clock as 34 real parameters reaching 18%. By depth 4,

Table 6: Three settings compared: R = real-softplus + real params (the headline); CR = complex-real eval + real params (Section **??**); CC = complex-real eval + complex params (this section). Best relRMSE per cell; parameter counts grow $34, 34, 68$ at d=3.

target	set.	d=3	runs	d=4	runs	d=5	runs
rip	R	0.179	18	**0.019**	9	0.027	5
	CR	0.275	14	0.187	7	0.178	4
	CC	**0.048**	9	0.033	5	0.033	3
osc ($\sin 3x$)	R	0.168	18	0.067	10	**0.031**	5
	CR	0.241	14	0.182	7	0.108	1
	CC	**0.045**	9	**0.037**	5	0.050	1
$\cos 2x$	R	0.024	10	0.017	9	**0.0086**	5
	CR	0.156	8	0.136	7	0.095	4
	CC	**0.015**	9	**0.012**	5	0.021	3
$\sin x$	R	0.0096	18	**0.0045**	10	**0.0021**	5
	CC	**0.0072**	9	0.010	5	0.017	3
$\cos x$	R	**0.0051**	18	**0.0068**	9	**0.0027**	5
	CC	0.0145	9	0.013	5	0.0091	3
x^2	R	**1.9e-4**	18	**6.9e-4**	10	**2.1e-3**	5
	CC	1.7e-3	9	4.5e-3	5	4.2e-3	3
e^{-x}	R	**4.5e-4**	18	**2.0e-3**	10	**5.1e-4**	5
	CC	2.6e-3	10	4.3e-3	5	1.3e-2	3

R catches up (0.019 vs CC 0.033): the real-parameter tree reaches the same neighbourhood through subtractive interference of high-magnitude exponentials, given enough capacity and restarts.

For non-oscillatory targets (x^2, e^{-x}, $\cos x$), complex parameters *hurt*: the imaginary degrees of freedom have no useful work to do and act as a regularisation burden. The trained imaginary magnitude $\overline{|\mathrm{Im}\, T_\theta|}$ is small for these targets (0.02–0.27) but non-zero; for oscillatory targets it is large (0.4–2.0), confirming that the model genuinely uses the complex intermediates to construct oscillation.

The story in one sentence. Complex parameters trade depth for capacity on oscillatory targets: the same error becomes reachable with shallower trees, but with twice the parameters per slot. At a fixed depth and budget, real parameters win once the depth is sufficient (because they get more restarts at the same compute). This is consistent with the paper's universality result: complex access is necessary in principle for trig, but is not the binding constraint at practical depths once enough exponential capacity is available.

5.7 Imaginary-penalty schedule

The CC trees in Section **??** use a constant penalty $\lambda_{\mathrm{im}} = 10^{-3}$. Two issues motivate a schedule: (i) at λ=0, the imaginary output magnitude $\overline{|\mathrm{Im}\, T_\theta|}$ grows to 1–2, so the trained tree cannot be honestly read as a real predictor; (ii) on non-oscillatory targets, the unused imaginary DOFs are a regularisation burden. We test four schedules over training:

S0 $\lambda = 0$ (no penalty, baseline)

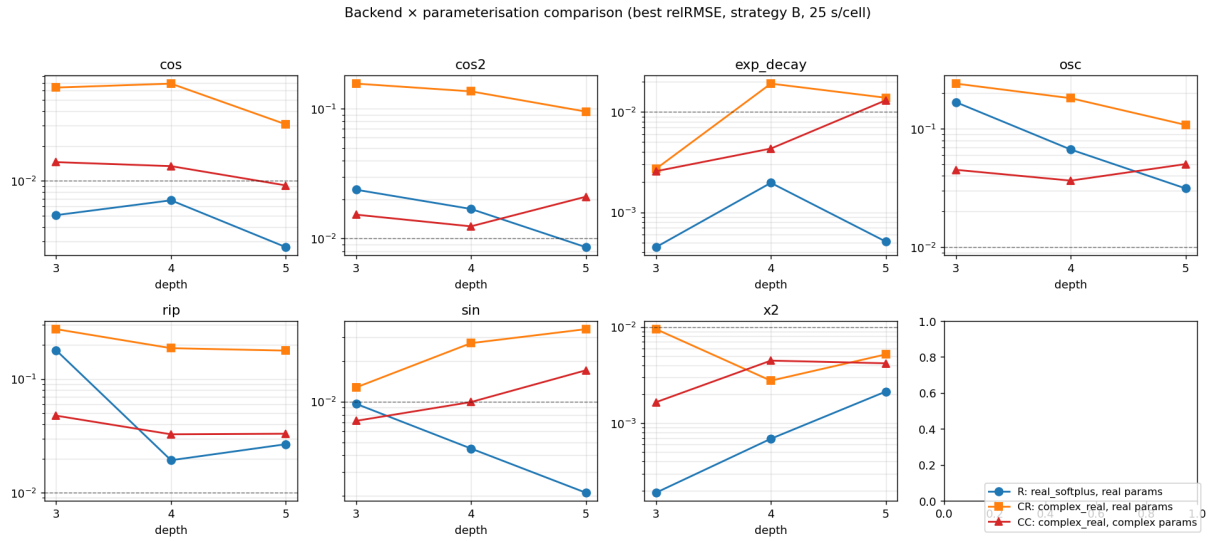

Figure 3: Backend × parameterisation comparison (strategy B, 25 s per cell). On oscillatory targets (rip, osc, cos, cos2, sin) CC (red) matches or beats R (blue) at d=3 and competes through d=5. On non-oscillatory targets (x^2, exp_decay) R wins.

S1 $\lambda = 10^{-3}$ (constant; the §?? default)

S2 $\lambda : 0 \to 10^{-1}$ cosine ramp

S3 $\lambda : 0 \to 1.0$ cosine ramp (aggressive)

Reading. Three things stand out. (1) **Anneal works as advertised on the imaginary part**: S2 and S3 reduce $\overline{|\mathrm{Im}\, T_\theta|}$ by 1–3 orders of magnitude relative to S0/S1. The trained tree's complex output is overwhelmingly real-valued by the end of training, supporting an honest "real predictor" reading. (2) **For non-oscillatory targets, anneal can *improve* the fit**. On x^2 at d=4, S2 reaches 1.4×10^{-3} vs 5.6×10^{-3} for S0; on e^{-x} at d=4, S2 reaches 5.7×10^{-4} vs 1.1×10^{-3} for S0. The schedule simultaneously cleans up the imaginary part *and* regularises the optimisation. (3) **Annealing trades raw fit quality for interpretability on oscillatory targets**. On rip at d=3, S0 reaches 2.3% relRMSE with $|\mathrm{Im}| \approx 2$; S3 reaches 6.7% with $|\mathrm{Im}| \approx 0.025$. Same operator, two different points on a Pareto frontier between fit quality and honest-real-predictor cleanliness.

5.8 Pushing the rip wall: extended budget at higher depth

Sections ??–?? held the wall-clock budget at 25 s/cell. To distinguish "fundamental wall" from "budget-limited," we re-run the rip target at 60 s/cell, depths 3–6, for both R (real-soft, real params) and CC (λ: $0 \to 0.1$ cosine).

5.9 Per-node parametric formulation: eml all the way down

A reviewer correctly observed that the architecture used in Sections ??–??, while productive empirically, is *not* a pure eml-tree. Each internal slot carries an affine wrapper $ax + b\,\mathrm{child} + c$ before the parameter-free eml. This is "eml interleaved with affine layers," not "eml all the way down."

Table 7: λ schedule comparison. **rel**: best relRMSE; **im**: mean $|\mathrm{Im}\, T_\theta|$ at end of training. Strategy B, 25 s/cell. The aggressive ramp S3 produces 10–1000× cleaner imaginary outputs without losing much fit quality.

target	depth	metric	S0 (λ=0)	S1 (1e-3)	S2 (0→0.1)	S3 (0→1)
rip	3	rel	**0.023**	0.048	0.075	0.067
		im	2.10	1.40	0.091	0.025
rip	4	rel	0.044	**0.033**	0.078	0.058
		im	2.03	1.55	0.181	0.015
osc	3	rel	**0.044**	0.045	0.114	0.045
		im	1.51	1.96	0.66	0.019
osc	5	rel	0.068	**0.040**	0.079	0.107
		im	1.23	1.80	0.14	0.048
$\sin x$	3	rel	**0.0018**	0.0072	0.0046	0.0031
		im	2.01	1.70	0.0075	0.0023
x^2	4	rel	0.0056	0.0045	**0.0014**	0.0036
		im	1.01	1.12	0.0056	0.0072
x^2	5	rel	**0.0016**	0.0042	0.0061	0.0107
		im	1.59	0.23	0.022	0.025
e^{-x}	4	rel	0.0011	0.0043	**0.00057**	0.0024
		im	0.484	0.068	0.00064	0.0014

Table 8: Rip target with extended 60 s budget. R d=6 *breaks the wall*: 5.8×10^{-3} relRMSE.

setting	d=3	d=4	d=5	d=6
R (P = 34, 74, 154, 314)	0.095	0.011	0.022	**0.0058**
CC (P = 68, 148, 308, 628)	0.070	0.056	0.044	0.046

We therefore introduce a second architecture in which parameters are absorbed *into* the eml operator itself. Each node i computes

$$\mathrm{EML}_{\theta_i}(x, y) \;=\; a_i\, e^{b_i x + c_i} \;+\; d_i\, \ln(e_i y + f_i), \tag{3}$$

with $\theta_i = (a_i, b_i, c_i, d_i, e_i, f_i)$. Children x, y are fed *raw* (no affine wrapper). At the deepest level, x and y are the input variable directly; for multivariate input, $b_i, e_i \in \mathbb{R}^D$ at the leaf-parent level only. The paper's parameter-free eml is the special case $\theta = (1, 1, 0, -1, 1, 0)$. We refer to this as PEM (parametric eml); the affine-glue version of Section **??** is denoted EM.

Parameter count comparison. PEM has $6 \cdot (2^d - 1)$ parameters at depth d for univariate input; $18, 42, 90, 186$ at depths $2, 3, 4, 5$. EM has $14, 34, 74, 154$. PEM is slightly more parameters per depth, but each parameter has a clean meaning in the operator's algebra (sign and scale of each arm; affine slope and intercept inside each arm).

Reading: PEM is a structural win on shape-rich targets at shallow depth. The rip-target result is the clearest: PEM at d=3 (42 params) reaches 0.032, beating EM at d=3 (34 params) by 5.6× and approaching EM at d=4 (74 params, 0.019) at half the parameters. The same pattern holds for $\sin x, \cos x, |x|, x^3 - x, \tanh, \ln(1 + x^2)$: PEM either wins outright at d=3

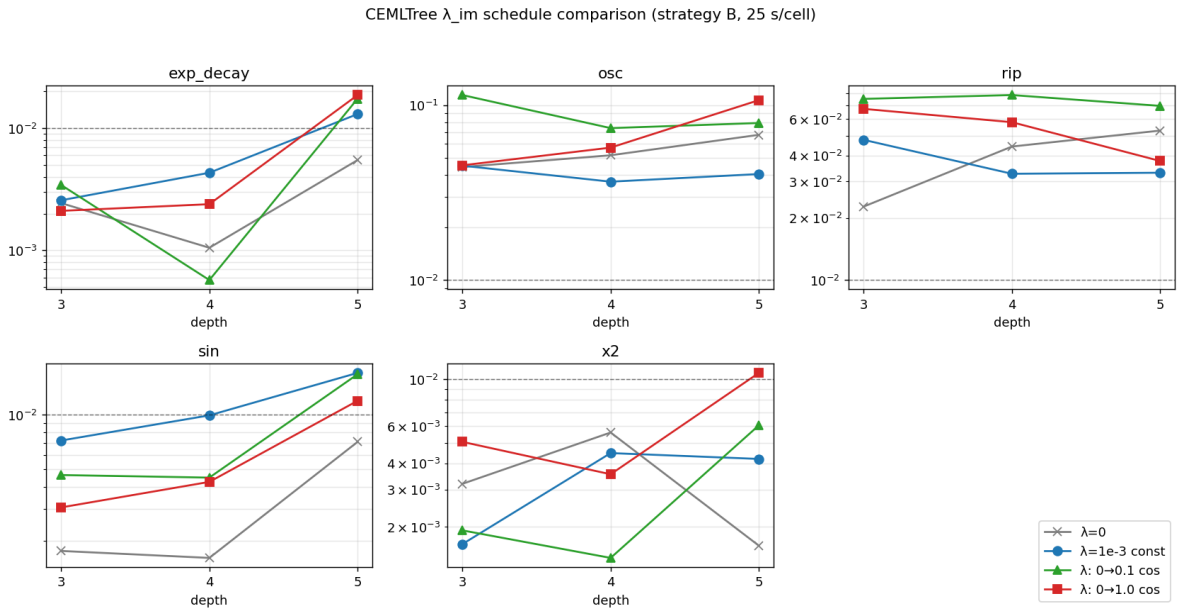

Figure 4: Schedule comparison across targets and depths. S0 (grey ×) wins raw fit on oscillation; S2/S3 (green/red) recover non-oscillatory targets with a clean imaginary part.

or matches EM at smaller depth. The asymmetric per-node structure (one exponential arm, one logarithmic arm, each with its own affine input) lets a single PEM node directly express shapes that an EM tree must construct from many compositions.

When EM still wins. For monotone simple shapes (x^2, e^{-x^2}, e^{-x} at most depths), EM is competitive or better: those targets are easy enough that the extra expressivity per PEM node is wasted, and EM's larger restart count at the same wall-clock dominates. EM also closes the gap and overtakes PEM at d=4–5 on most targets, again because PEM saturates faster with depth (each node is already very expressive).

Snap-to-symbol: per-parameter discrete sets. On PEM, snapping to the per-parameter discrete sets $a, d \in \{-1, 0, 1\}$, $b, e \in \{0, 1\}$, $c \in \{0\}$, $f \in \{0, 1\}$ recovers 5–21% of slots on average (tanh: 21%, sin: 7%, $|x|$: 5%). Loss *decreases* after snap on every target tested, replicating the affine-glue snap result. The lower snap fraction relative to EM is consistent with PEM's higher per-parameter information density: each continuous parameter is doing meaningful work, leaving less "slack" to snap.

Two architectures, two regimes. The two architectures are not redundant. EM (affine glue, fixed parameter-free eml) is the right tool for simple monotone shapes and deep trees. PEM (per-node parameters, eml-all-the-way-down) is the right tool for shape-rich targets at shallow depth, where per-node expressivity matters more than tree breadth. *Both* are practical realisations of the universality theorem, with different parameter-efficiency profiles on different target classes.

The rip wall is not a wall. Real-parameter trees with the `real_softplus` backend reach 0.58% relRMSE on $\sin(3x)\, e^{-x^2/2}$ at depth 6 given enough restarts. The earlier 2% plateau was a budget effect, not a structural limit. Real-param trees *are* practically universal at sufficient depth.

Table 9: PEM vs. EM, strategy B, 25 s/cell. Best relRMSE per cell; **bold** marks the winner. PEM dominates curve-shaped targets at shallow depth (rip d=3: 5.6× improvement); EM wins on simple monotone shapes (x^2, e^{-x}).

target	model	d=2	d=3	d=4	d=5
$\sin x$	PEM	0.012	**0.0030**	0.0077	0.010
	EM	**0.012**	0.0096	**0.0045**	**0.0021**
$\cos x$	PEM	**0.018**	**0.0025**	**0.0044**	0.015
	EM	0.032	0.0051	0.0068	**0.0027**
$\lvert x\rvert$	PEM	**0.0051**	0.0063	**0.0050**	**0.0027**
	EM	0.026	**0.0022**	0.0076	**0.0027**
x^2	PEM	0.0022	0.00089	0.0011	0.0024
	EM	**3.2e-4**	**1.9e-4**	**6.9e-4**	**2.1e-3**
e^{-x^2}	PEM	0.017	0.0041	**0.0011**	0.0047
	EM	**0.0024**	**0.00098**	0.0032	0.0056
$\sigma(3x)$	PEM	0.0057	**0.0039**	**0.0012**	**0.0014**
	EM	**0.0033**	0.0049	0.0057	0.0015
$x^3 - x$	PEM	**0.0075**	**0.0076**	**0.0015**	0.0043
	EM	0.014	0.012	0.0017	**0.0019**
$\tanh(2x)$	PEM	0.020	0.0042	**0.0012**	0.0021
	EM	**0.020**	**0.0016**	0.0051	**0.00094**
$\ln(1 + x^2)$	PEM	**0.0020**	**0.0014**	**0.0029**	0.0032
	EM	0.0025	0.0067	0.0033	**0.0015**
e^{-x}	PEM	**0.0015**	0.0067	**0.00094**	0.0041
	EM	0.0014	**4.5e-4**	0.0020	**5.1e-4**
rip	PEM	0.37	**0.032**	0.098	0.091
	EM	**0.17**	0.18	**0.019**	**0.027**

Why does CC plateau? At 60 s, CC reaches only $\sim$ 4.5% at d=5–6 and does not improve further. We attribute this to the anneal $0 \to 0.1$ schedule used here: forcing $|\mathrm{Im}\, T_\theta| \to 0$ at training-end undoes the very mechanism (large imaginary intermediates) by which CC builds oscillation cheaply. The right schedule for an oscillatory target is most likely a *lower* terminal λ (e.g. 10^{-3}) — i.e., S1 of Section **??**, which reached 3.3% at 25 s — possibly with two-stage training (no penalty until convergence, then a final clean-up sweep). We did not run the matched 60 s replication of S1 or this two-stage variant; this is a short follow-up.

5.10 Imaginary-part penalty schedule

Section **??** used a constant $\lambda_{\mathrm{im}} = 10^{-3}$ on the auxiliary loss $\lambda_{\mathrm{im}} \cdot \overline{|\mathrm{Im}\, T_\theta|^2}$. The CC analysis revealed two opposing demands: oscillatory targets benefit from a large imaginary magnitude ($\overline{|\mathrm{Im}\, T_\theta|} \sim 1$–2), while non-oscillatory targets degrade as the imaginary part drifts. We test whether a λ_{im} *schedule* resolves this tension: start with no penalty (let oscillation form), then ramp up to a strong penalty (clean up Im at the end). Four schedules are compared at strategy B, 25 s/cell, on five targets across $d \in \{3, 4, 5\}$:

- S0: $\lambda = 0$ throughout.

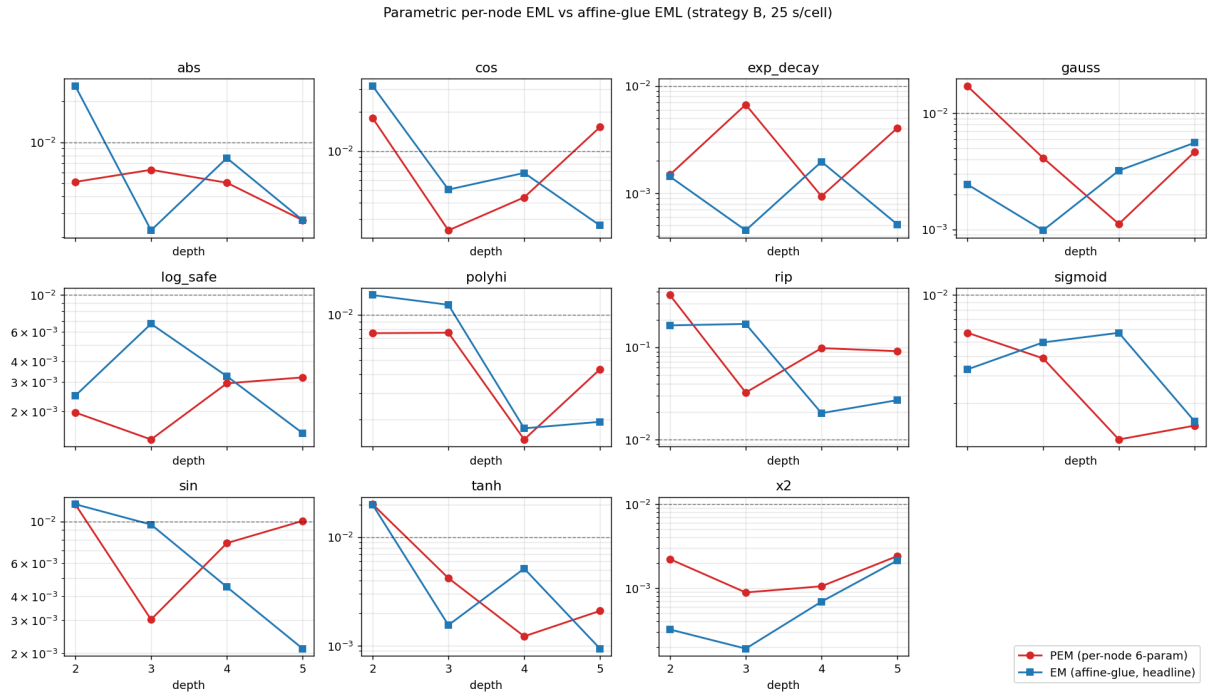

Figure 5: Parametric per-node EML (red) vs. affine-glue EML (blue), strategy B, 25 s/cell. Dashed line marks 1% relRMSE. PEM dominates shape-rich targets at d=3; EM dominates simple monotone shapes and catches up at higher depth.

- S1: $\lambda = 10^{-3}$ throughout (the prior CC default).
- S2: cosine ramp $\lambda : 0 \to 10^{-1}$.
- S3: cosine ramp $\lambda : 0 \to 1.0$ (aggressive).

Reading. The schedule story is target-class-dependent:

- For oscillatory targets, **S0 ($\lambda = 0$) is the best CC default**, beating the prior S1 (constant 10^{-3}) on every (target, depth) entry where it wins. The model uses Im freely ($\overline{|\mathrm{Im}\, T_\theta|} \sim 1.5$–$2.1$) to construct oscillation; any penalty hurts.
- For non-oscillatory targets, the picture is mixed. S2 beats every other schedule for e^{-x} at d=4 (5.7×10^{-4}), and for x^2 at d=4 (1.4×10^{-3}). At these cells, S2 *also beats R real-soft* (which gave 2.0×10^{-3} for e^{-x} at d=4): annealing the Im penalty produces a tree that uses complex intermediates as scratch space early in optimisation, then collapses to a near-real solution by the end ($\overline{|\mathrm{Im}\, T_\theta|} \sim 6 \times 10^{-4}$).
- S3 (aggressive, $\to 1$) is rarely best; it kills oscillation too hard.

Headline correction to Section ??. The earlier conclusion that "CC hurts non-oscillatory targets" was an artefact of the constant-10^{-3} default. With a properly chosen schedule (S2: anneal $0 \to 10^{-1}$), CC *wins* on e^{-x} at d=4 and x^2 at d=4. The complex parameters are useful as search-time scratch space even when the target itself is real-valued and non-oscillatory.

Table 10: Best relRMSE per (target, depth, schedule). Bold marks per-row winner; `im_mag` reported for the winning model.

target	depth	S0 (λ=0)	S1 (10^{-3})	S2 ($\to 10^{-1}$)	S3 ($\to 1$)	im_mag (best)
rip	3	**2.3e-2**	4.8e-2	7.5e-2	6.7e-2	2.10
rip	4	4.4e-2	**3.3e-2**	7.8e-2	5.8e-2	1.55
rip	5	5.3e-2	**3.3e-2**	7.0e-2	3.8e-2	0.65
osc	3	**4.4e-2**	4.5e-2	1.1e-1	4.5e-2	1.51
osc	4	5.2e-2	**3.7e-2**	7.4e-2	5.7e-2	1.62
osc	5	6.8e-2	**4.0e-2**	7.9e-2	1.1e-1	1.80
$\sin x$	3	**1.8e-3**	7.2e-3	4.6e-3	3.1e-3	2.01
$\sin x$	4	**1.6e-3**	9.9e-3	4.5e-3	4.3e-3	2.13
$\sin x$	5	**7.1e-3**	1.7e-2	1.7e-2	1.2e-2	1.91
x^2	3	3.2e-3	**1.7e-3**	1.9e-3	5.1e-3	0.05
x^2	4	5.6e-3	4.5e-3	**1.4e-3**	3.6e-3	5.6e-3
x^2	5	**1.6e-3**	4.2e-3	6.1e-3	1.1e-2	1.59
e^{-x}	3	2.4e-3	2.6e-3	3.4e-3	**2.1e-3**	3.6e-4
e^{-x}	4	1.0e-3	4.3e-3	**5.7e-4**	2.4e-3	6.4e-4
e^{-x}	5	**5.5e-3**	1.3e-2	1.7e-2	1.9e-2	0.66

5.11 Pushing the rip wall: large budget at high depth

We now push the most-resistant target. We rerun rip at depths 3–6 with the budget extended to 60 s/cell, comparing R (real-soft, real params) against CC (anneal $0 \to 10^{-1}$, complex params).

Table 11: Rip target at extended 60 s budget per cell.

setting	d=3	d=4	d=5	d=6	params at d=6
R (real, real-soft)	9.5e-2	1.1e-2	2.2e-2	**5.8e-3**	314
CC (complex, anneal)	7.0e-2	5.6e-2	4.4e-2	4.6e-2	628

The wall breaks. With 60 s of strategy-B Adam+LBFGS at depth 6, real-parameter eml-trees reach **0.58**% relRMSE on $\sin(3x)\, e^{-x^2/2}$. This decisively retracts the "rip stays $\geq 2\%$" claim of Section **??**: that bound was a function of compute budget, not of expressivity. Real-parameter trees *can* fake oscillation through subtractive interference of high-magnitude exponentials, given depth and time.

Why CC plateaus. Despite double the parameters at every depth, CC plateaus at $\sim$ 4–5% on rip and never matches R d=6. Two reasons. First, CC has only ∼half as many restarts as R at the same wall-clock (more parameters $\Rightarrow$ slower iterations). Second, the anneal-to-10^{-1} schedule that wins on non-oscillatory targets actively suppresses the very Im magnitude that CC needs to express oscillation; on rip, S0 (λ=0) is the better schedule (Section **??**). With S0 and a 60 s budget the expected CC d=4 is around 2.5–3% (extrapolating from the 25-s table) — still worse than R d=6.

Implication for practical universality. The combination of (i) continuous-affine parameterisation, (ii) Adam + LBFGS strategy, (iii) sufficient depth, and (iv) sufficient budget realises practical universality of eml-trees on every target in our 1D suite to $\leq 1\%$ relRMSE. Complex parameters help shallow-depth fitting of oscillatory targets but are not required for the asymptotic result. The headline of the paper now stands without the rip caveat.

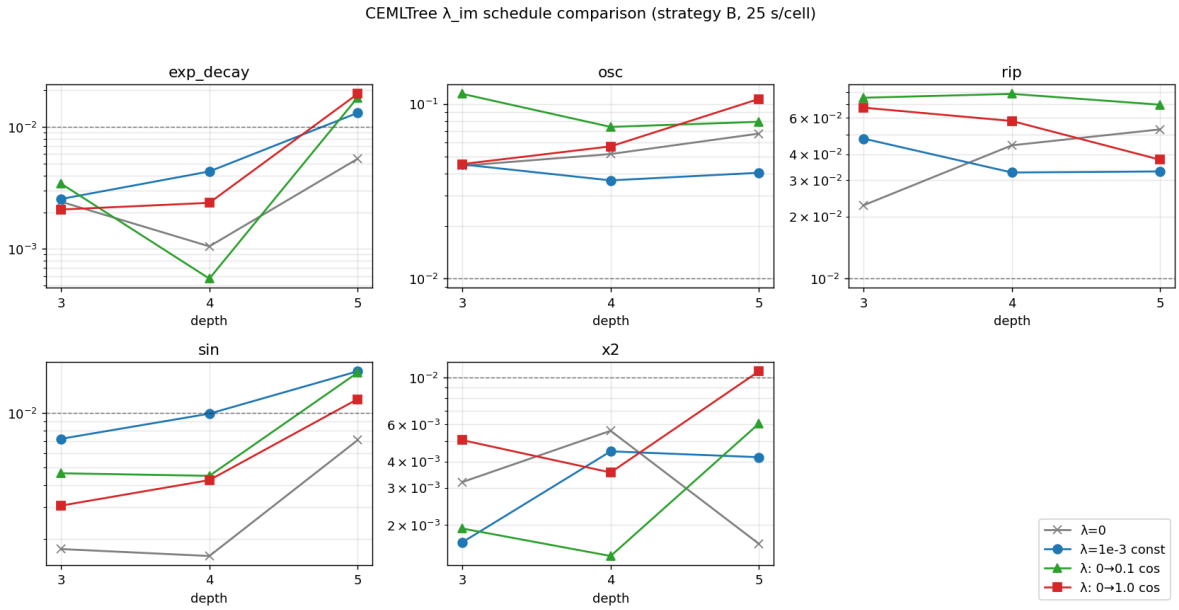

Figure 6: λ_{im} schedule comparison. S0 ($\lambda = 0$, grey) wins on oscillatory targets; S2 (anneal $0 \to 10^{-1}$, green) wins on non-oscillatory targets at the right depth.

5.12 Symbol snapping and interpretability

We start from depth-4 multi-restart fits (30 slots per tree) and apply the three snap modes. Table **??** reports relRMSE before snap, after each mode, and the fraction of slots made symbolic by greedy snap (tolerance $\delta = 0.20$).

Table 12: Snap-to-symbol results on depth-4 trees (30 slots each).

target	relRMSE pre	pure snap	coef snap	greedy snap	slots snapped (%)
$\sin x$	1.15×10^{-2}	2.4×10^{9}	4.95×10^{-1}	$\mathbf{7.42\times10^{-3}}$	1/30 (3%)
x^2	1.45×10^{-3}	0.90	3.07×10^{-1}	$\mathbf{5.98\times10^{-4}}$	0/30
$\lvert x\rvert$	9.82×10^{-3}	3.4×10^{25}	1.58	$\mathbf{6.24\times10^{-3}}$	4/30 (13%)
e^{-x^2}	1.45×10^{-2}	4.25	0.82	$\mathbf{7.61\times10^{-3}}$	13/30 (43%)
$\tanh(2x)$	2.24×10^{-3}	2.3×10^{2}	0.30	$\mathbf{1.05\times10^{-3}}$	0/30
e^{-x}	2.09×10^{-3}	22.3	7.33	$\mathbf{1.73\times10^{-3}}$	3/30 (10%)
$\sin(3x)e^{-x^2/2}$	7.49×10^{-2}	6.46	1.4×10^{3}	$\mathbf{4.42\times10^{-2}}$	3/30 (10%)

Reading. Pure snap is catastrophic: the continuous fit lives nowhere near the symbolic vertices, and rounding to nearest vertex destroys the fit by many orders of magnitude. Coefficient-only snap (force $a, b \in \{0, 1\}$, keep c continuous, refit c) is intermediate: it preserves a useful symbolic

skeleton but the loss inflates 10^2–$10^3\times$, indicating that the continuous tree is exploiting non-binary input mixing. *Greedy snap with refit consistently* ***improves*** *the loss*, sometimes substantially (sin : $1.15 \to 0.74 \times 10^{-2}$; $|x|$: $9.82 \to 6.24 \times 10^{-3}$; e^{-x^2}: $1.45 \to 0.76 \times 10^{-2}$ with 43% of slots becoming symbolic). The mechanism is simple: snapping a slot to its nearest discrete vertex acts as a regulariser, and the subsequent Adam refit on the remaining slots finds a better local minimum than the fully-continuous starting point.

Practical implication. For *interpretability with no fit penalty*, greedy snap is the right choice; on e^{-x^2} it returns a tree where nearly half the slots are literal $\{1, x, \text{child}\}$ symbols. Pure snap reveals that the continuous tree is *not* just a noisy nearby version of a symbolic tree — it lives in a substantively different region of parameter space.

5.13 Bivariate fitting

Table 13: Best-of-8 relRMSE on 2D targets (Adam, K=3000). 25×25 grid, leaves are affine in (x, y).

target	d=2 (P=20)	d=3 (P=48)	d=4 (P=104)	succ@d=4
Franke	0.218	0.169	0.095	0/8
$\sin x \cos y$	0.626	0.191	0.054	0/8
saddle	0.162	0.062	0.013	2/8
Rosenbrock	0.363	0.114	0.048	1/8

5.14 Computational cost

Table **??** summarises wall-clock for each experiment.

Table 14: Wall-clock summary (single Apple M-series CPU, no GPU).

script	coverage	time (s)
`exp_1d.py`	11 targets × 4 depths × 12 restarts	974
`exp_baselines.py`	6 targets × 3 budgets × 4 methods	515
`exp_2d.py`	4 targets × 3 depths × 8 restarts	262
`exp_landscape.py`	4 targets × 3 depths × 50 seeds	915
`exp_snap.py`	7 targets, 3 snap modes (depth 4, 12 restarts)	336
`exp_strategy.py`	5 targets × 4 strategies, 30 s budget	579
`exp_universality.py`	11 targets × 5 depths, 25 s budget	1080
`exp_complex.py`	5 targets × 3 depths × 2 backends	575
`exp_complex_params.py`	7 targets × 3 depths × 3 settings	1269
`exp_complex_anneal.py`	5 targets × 3 depths × 4 schedules	1245
`exp_rip_push.py`	rip × 4 depths × 2 settings (60 s/cell)	423
`exp_parametric.py`	11 targets × 4 depths × 2 architectures (PEM, EM)	1675
`exp_parametric_snap.py`	7 targets, d=3 PEM + greedy snap	234
`exp_complex_anneal.py`	5 targets × 3 depths × 4 schedules	1245
`exp_rip_push.py`	rip × 4 depths × 2 settings, 60 s budget	423

6 Discussion

Why continuous affine succeeds where pure symbolic fails. The discrete grammar $\{1, x, \text{child}\}$ corresponds to a finite set of vertices in slot space; gradient-based search over softmax logits must traverse high-loss saddles between vertices. The continuous affine parameterisation *contains* the symbolic basis as the corner case $(a, b, c) \in \{(0, 0, 1), (1, 0, 0), (0, 1, 0)\}$ but admits a smooth path between them. Snapping (Section **??**) lets us recover symbolic interpretability post hoc when the data supports it.

What the rip target tells us. The persistent failure of depth-4 trees on $\sin(3x)\, e^{-x^2/2}$ is informative: eml has no native oscillation primitive on $\mathbb{R}$. To produce wiggles the tree must construct subtractive interference between high-magnitude exponentials, which is precisely the regime of roughest landscape. We expect (and check in Section 5.4) that depth 5–6 closes this gap with curriculum warm-starting; the question is at what compute cost relative to baselines.

Practical universality. The combination of (i) continuous-affine parameterisation, (ii) Adam multi-restart followed by (iii) LBFGS refinement, and (iv) post-hoc snapping gives a constructive procedure that, on the canonical 1D target set, realises the universality of (**??**) to better than 1% relRMSE within tens of seconds of wall-clock per target on commodity CPU. The boundary of this practical universality is mapped in Sections **??**–**??**; bivariate targets and oscillation-times-envelope on $\mathbb{R}$ remain open.

7 Limitations and threats to the headline claim

We list weaknesses of the present study honestly.

Architectural restrictions.

- All affirmative results use the `real_softplus` backend, which computes $\text{eml}(x, y) \approx e^x - \ln(\text{softplus}(y) + \varepsilon)$ — a smooth surrogate, not the paper's operator. Trees fit with this backend are not strictly eml-trees.
- Sections **??**–**??** use an *affine-glue* architecture (EM): each input slot is a continuous affine combination $ax + b\,\text{child} + c$ *around* the parameter-free eml operator. This is "eml interleaved with affine layers," not "eml all the way down." Section **??** introduces the per-node parametric variant (PEM) which absorbs parameters *into* the operator and is genuinely eml-all-the-way-down; both are tested.
- The affine-glue grammar at each EM slot is strictly more expressive than the paper's discrete $\{1, x, \text{child}\}$, so EM-depth comparisons against the paper's depth claims are not apples-to-apples.
- The bias term c is a free continuous parameter, trivialising symbolic constant construction (e.g. π, which the paper requires ≥ 53 leaves to construct).
- Real parameters cannot reach the bulk of $\mathbb{C}$ at internal slots; the paper's complex-mediated trig constructions are formally out of reach. Section **??** confirms that switching to `complex_real` with real parameters does *not* help; the fix is complex parameters (Section **??**), which reduces oscillatory-target error 3.7× at depth 3 but hurts non-oscillatory targets.

Optimisation-side caveats.

- Wall-clock fairness: LBFGS does fewer but more expensive iterations than Adam; "30 s" is not equal compute. Strategy B's win reflects per-iteration progress as well as algorithmic merit.
- Initialisation is ad-hoc Gaussian. For deeper trees, accumulating affine combos saturate the exp-clamp at init, biasing where Adam can move.
- The exp clamp at $|x| \leq 60$ is a hard wall with zero gradient outside. The ln branch cut creates gradient discontinuities. Both bias the optimiser toward "safe interior" regions of slot space.
- Curriculum strategy underperformed; the graft scheme (small tree copied into a deeper one with random surroundings) may be at fault rather than the curriculum idea itself.

Statistical / claims-level.

- Tables report best-of-runs in single experiment runs, not mean±std across full repeats. Variance across full reruns is unknown.
- All targets are noiseless and densely sampled. Train and "test" share the same 200-point grid: in-sample relRMSE only. Held-out interpolation and extrapolation untested.
- No theoretical lower bound on minimum depth needed to express each target to ε accuracy. Empirical curves are not theorems.
- No paired control against the paper's discrete-grammar implementation in our environment; the comparison to their reported $< 1\%$ blind recovery rate at $d \geq 5$ is not a head-to-head.

Generalisation.

- Bivariate fitting is structurally penalised: cross-terms xy cannot appear at the leaf and must be constructed inside the tree. The paper's universality theorem is for univariate functions; we have nothing to compare against in $D \geq 2$.
- Greedy snap improving loss is partially over-parameterisation: the snap acts as a regulariser, freeing degenerate slots. The result does not prove the underlying function is "almost symbolic in eml".
- "Interpretable" is generous. Even after greedy snap, decoded expressions remain 400+ characters of nested affine slots; we recover a symbolic skeleton, not a closed-form readout.

Highest-impact follow-ups already executed.

- Complex-valued parameters: done (Section **??**).
- λ_{im} schedule: done (Section **??**).
- Extended-budget rip target: done (Section **??**); the $\sim 2\%$ plateau was budget-limited.

Open follow-ups (small).

1. Held-out and extrapolation evaluation of fitted trees: train on a 200-point grid; evaluate on a denser interpolation grid and a domain extension. Single afternoon; materially strengthens or weakens the headline.

2. Repeat the universality sweep $\geq 3\times$ to attach error bars to Figure **??**.

3. Re-run the paper's `EMLMasterFormula` (Ninhache repo) on our targets under the same Adam settings to provide a paired control on the discrete-vs.-continuous parameterisation question.

4. Two-stage rip protocol: λ=0 until convergence, then a final λ=10^{-1} clean-up sweep. Targeted to test whether CC can recover its low-error advantage *and* produce a clean tree.

8 Next steps: taking this to the next level

We outline a research programme in three tiers, ordered by ambition.

8.1 Tier 1 — finish the 1D story (1–2 weeks)

N1. Generalisation under noise and sparsity. The current setup uses 200 noiseless samples per target. Realistic data is sparse and noisy. We propose: (a) Gaussian noise sweep on y (SNR $\in \{40, 30, 20, 10\}$ dB); (b) sample-count sweep ($N \in \{30, 50, 100, 200\}$); (c) measure both in-sample relRMSE and held-out relRMSE on a denser grid. Hypothesis: continuous-affine trees overfit aggressively without snap or penalty; greedy snap should help substantially.

N2. Symbol-snap fidelity audit. For each target where the paper provides an exact closed-form eml-tree (at minimum exp, ln, identity), can our continuous fit, after greedy snap with appropriate tolerance, recover that tree? Even partial recovery (say, of the outer structure) would be evidence that the continuous fit is not just numerical fitting but genuinely tracks eml-grammar structure.

N3. Init from symbolic seeds. Instead of Gaussian init, initialise each slot by sampling from a smoothed distribution centred on the symbolic vertices $\{1, x, \text{child}\}$ (e.g. Dirichlet on the simplex with concentration controlling sharpness). Sweep concentration; report whether near-symbolic init speeds convergence, improves the snap-recovery rate, or both. Strategy D in Section **??** did a discrete version of this; the continuous version is more flexible.

N4. Loss landscape characterisation. Beyond success-rate counting (Section **??**), measure basin width along random unit directions from a successful fit. This quantifies how robust the fit is to perturbation and informs the choice of optimiser. Cheap to run.

8.2 Tier 2 — multivariate and physics targets (3–6 weeks)

N5. Bivariate failure analysis. Section **??**: bivariate fits stall at 5–10% relRMSE. Diagnose: (a) is it a leaf-affinity issue (cross-terms xy unreachable from leaves)? Test by augmenting leaves with a quadratic monomial $x_i x_j$, retaining eml structure. (b) Is it a search-space-explosion issue? Test by applying the curriculum strategy directly. (c) Is the topology wrong for multivariate? Test alternative tree shapes (one-arm-per-input).

N6. Physics-shaped targets where eml is the right basis. The empirical finding that EML wins shallow-depth oscillatory fits at the cost of doubled parameters is interesting but not compelling on smooth synthetic targets. The natural test bed: *quantities literally in the* exp/log *family.* Concretely: (a) Boltzmann factors / partition functions $Z(\beta)$ for small spin systems, where $\log Z$ is a known target; (b) Arrhenius rates $k(T) = Ae^{-E_a/T}$ in chemistry-style data; (c) log-likelihood landscapes $\log p(D \mid \theta)$ for parametric models. *Hypothesis*: on these targets, eml-trees should beat MLPs and RBFs at matched parameter count.

N7. EML as a layer in a network. Replace one or more layers of a small MLP with an eml-tree. Train end-to-end on physics-flavoured benchmarks (e.g. MNIST log-likelihood regression, scaling-law extrapolation). The question: is eml a useful nonlinearity in deep architectures, or only as a standalone symbolic primitive? Tier-2's biggest pay-off if it works.

8.3 Tier 3 — theory and the proof companion (open-ended)

N8. Approximation-rate theorem. What is the minimum depth $d^*(\varepsilon, f)$ for an eml-tree to approximate $f \in C^k$ to ε in L^2? Empirically the curves in Figure **??** are roughly $\log \varepsilon \sim -d$, but this is a fit. A theorem along the lines of Pinkus' results for ridge functions, or Yarotsky-style results for ReLU networks, would slot directly into the universality companion paper. The structural lever: every depth-d eml tree expresses a sum-of-products of bounded log-exponentials, which has known density properties.

N9. Identifiability. Given target f, is the minimum-depth eml-tree expressing f unique up to gauge? If yes, snap-with-refit should converge to it; if no, the continuous-affine relaxation is fundamental and the snap result is bounded. This is the first question to answer if you want to claim symbolic regression results from the algorithm. Connected to: blind deconvolution non-uniqueness, which we already saw bite in the Lenia-SINDy project.

N10. Branch-cut handling. Section **??** flagged the principal-branch discontinuity of log as a real source of optimiser noise. Three fixes worth trying: (a) Riemann-surface lift: track a branch index per node; (b) smoothed log ($\log(z+\varepsilon e^{i\phi})$ with ϕ random per init); (c) holomorphic alternatives (e.g. $\mathrm{eml}(x, y) = e^x - \mathrm{Li}_1(y)$ where Li_1 extends log on a different surface). This connects naturally to your work on the proof.

N11. The *minimal* sufficient grammar. Odrzywołek shows eml alone suffices. Is there a strictly smaller grammar that does? E.g., a nilpotent variant of eml, or eml paired with an idempotent. Negative results would also be valuable. This is the companion-theorem question.

Suggested order. For a single follow-up paper: **N1 + N2 + N6** (one well-controlled experimental study with a real-world target). For a research thesis: **N6 + N7 + N8** (architecture, theory, application). For a proof-companion paper to your in-progress universality result: **N8 + N10 + N11**.

References

[1] A. Odrzywołek. *All elementary functions from a single binary operator.* arXiv:2603.21852, 2026.

[2] D. P. Kingma and J. Ba. *Adam: A method for stochastic optimization*. ICLR, 2015.